

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.1

Aim: Installation of JDK and compiling a simple Java Program

Requirements:

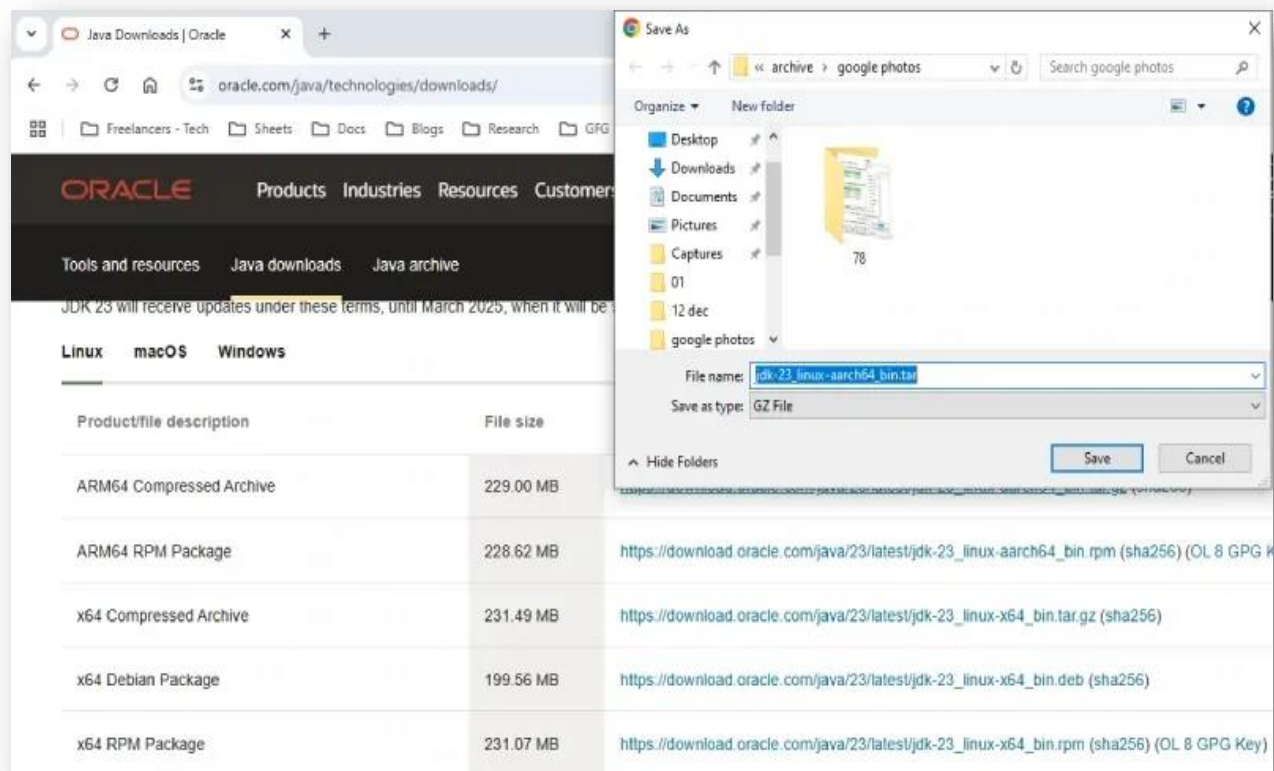
1. Access to a computer with an internet connection.
2. Basic understanding of command-line operations.

Download JDK (Java Development Kit)

We need to download the JDK first, find the steps below to get started:

Step 1: Visit the Official Website

Go to the [Official JDK website](https://www.oracle.com/in/java/technologies/javase-downloads.html) to download the file.

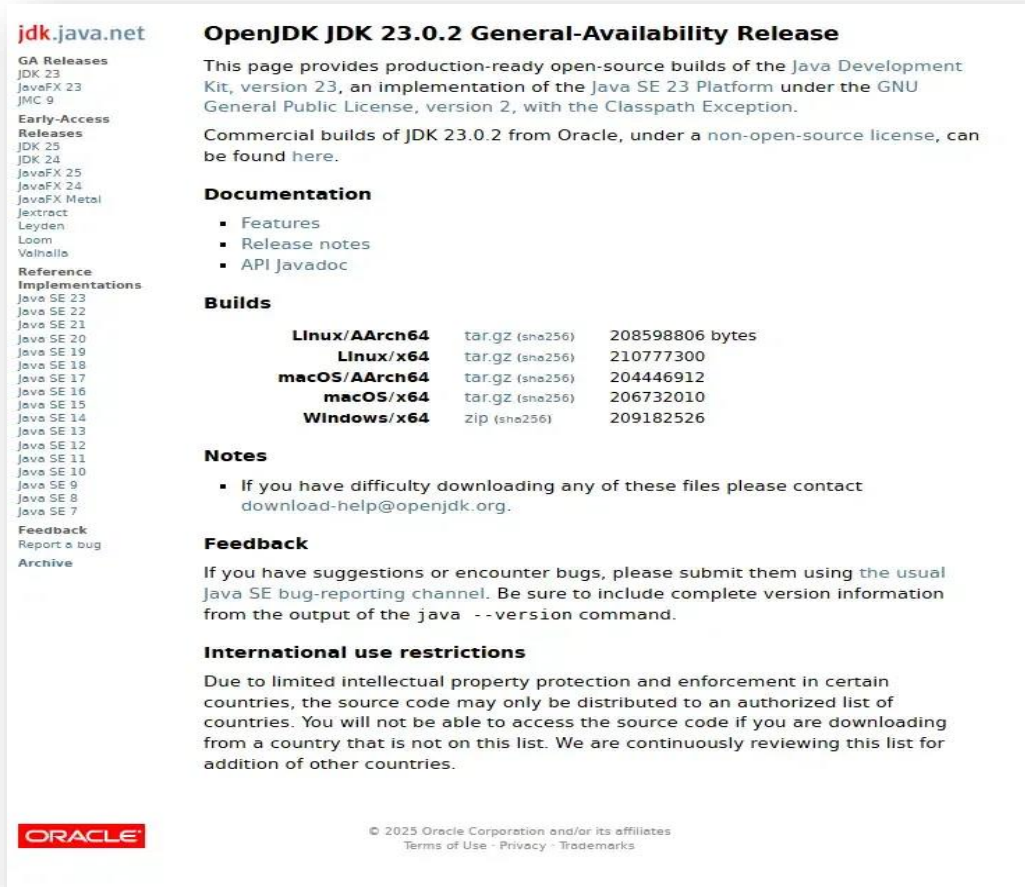


Since JDK is an open-source, so you can get it from [OpenJDK](https://openjdk.org/) in a few clicks.

~ 1 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA



The screenshot shows the OpenJDK JDK 23.0.2 General-Availability Release page. On the left is a navigation menu with links like GA Releases, JDK 23, JavaFX 23, JMC 9, Early-Access, Releases, JDK 25, JDK 24, JavaFX 25, JavaFX 24, JavaFX Metal, Jextract, Leyden, Loom, Valhalla, Reference Implementations, Java SE 23, Java SE 22, Java SE 21, Java SE 20, Java SE 19, Java SE 18, Java SE 17, Java SE 16, Java SE 15, Java SE 14, Java SE 13, Java SE 12, Java SE 11, Java SE 10, Java SE 9, Java SE 8, Java SE 7, Feedback, Report a bug, and Archive. The main content area has the title 'OpenJDK JDK 23.0.2 General-Availability Release'. It states that this page provides production-ready open-source builds of the Java Development Kit, version 23, an implementation of the Java SE 23 Platform under the GNU General Public License, version 2, with the Classpath Exception. It also mentions that commercial builds of JDK 23.0.2 from Oracle are available under a non-open-source license. Below this is a 'Documentation' section with links to Features, Release notes, and API Javadoc. A 'Builds' section follows, containing a table of download links and sizes for various operating systems and architectures. The table lists Linux/AArch64, Linux/x64, macOS/AArch64, macOS/x64, and Windows/x64, each with tar.gz and zip download options and their respective sizes in bytes. A 'Notes' section contains a link to download-help@openjdk.org. A 'Feedback' section provides information on how to report bugs. An 'International use restrictions' section explains that source code distribution is limited to certain countries. At the bottom, there is an Oracle logo and copyright information for 2025.

OpenJDK JDK 23.0.2 General-Availability Release

This page provides production-ready open-source builds of the Java Development Kit, version 23, an implementation of the Java SE 23 Platform under the GNU General Public License, version 2, with the Classpath Exception.

Commercial builds of JDK 23.0.2 from Oracle, under a non-open-source license, can be found here.

Documentation

- Features
- Release notes
- API Javadoc

Builds

Linux/AArch64	tar.gz (sha256)	208598806 bytes
Linux/x64	tar.gz (sha256)	210777300
macOS/AArch64	tar.gz (sha256)	204446912
macOS/x64	tar.gz (sha256)	206732010
Windows/x64	zip (sha256)	209182526

Notes

- If you have difficulty downloading any of these files please contact download-help@openjdk.org.

Feedback

If you have suggestions or encounter bugs, please submit them using the usual Java SE bug-reporting channel. Be sure to include complete version information from the output of the `java --version` command.

International use restrictions

Due to limited intellectual property protection and enforcement in certain countries, the source code may only be distributed to an authorized list of countries. You will not be able to access the source code if you are downloading from a country that is not on this list. We are continuously reviewing this list for addition of other countries.

ORACLE

© 2025 Oracle Corporation and/or its affiliates.
Terms of Use - Privacy - Trademarks

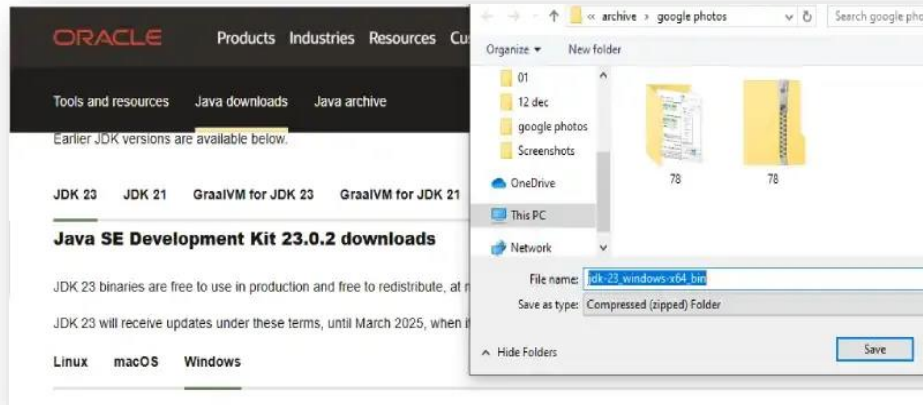
Step 2: Select the Appropriate Version:

As of 2025, the latest stable versions are JDK 23 (SE) and JDK 21 (LTS). Select the compatible version as per your operating system (Windows, Mac or Linux)

~ 2 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA



Step 3: License & Agreement

Go through all the License and Agreement before downloading (from Oracle website), it will not ask if you'll download it from OpenJDK website.

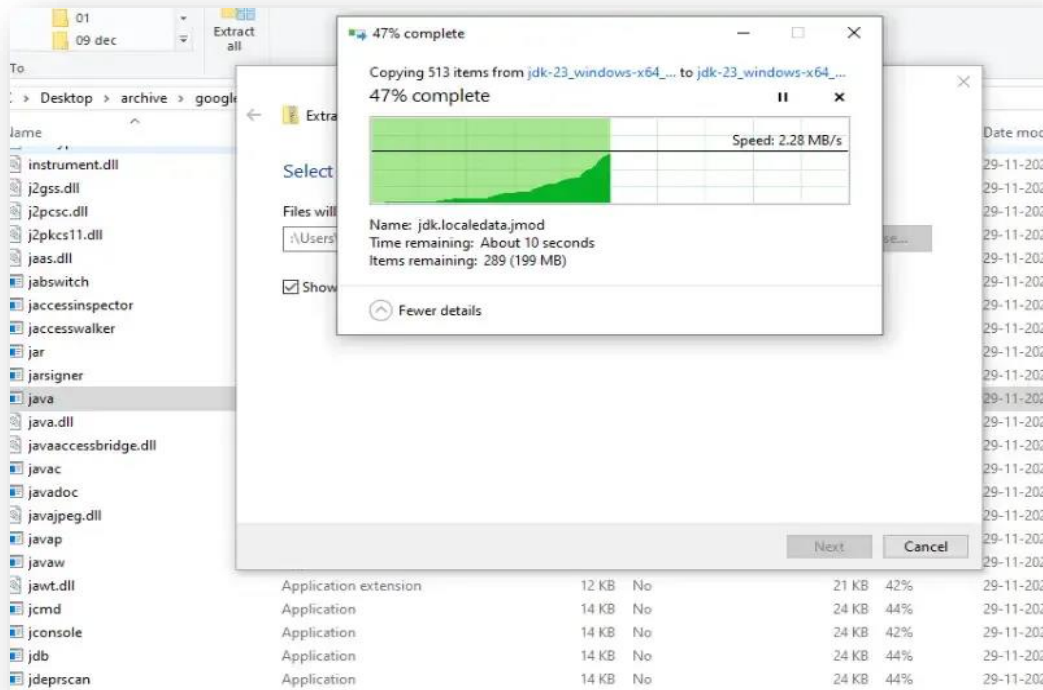
Install JDK on Windows

Follow the below steps to install JDK on Windows environment *i.e.* **Windows 7, Windows 8, Windows 8.1, Windows 10, and Windows 11.**

Step 1: Run the Java Development Kit (JDK) Installer

Locate the downloaded .exe file (e.g. **jdk-23-windows-x64_bin.exe**) and make the double click to begin the Installation process. Follow the installation wizard prompts to complete the installation process.

Practical file of Object-oriented programming using JAVA

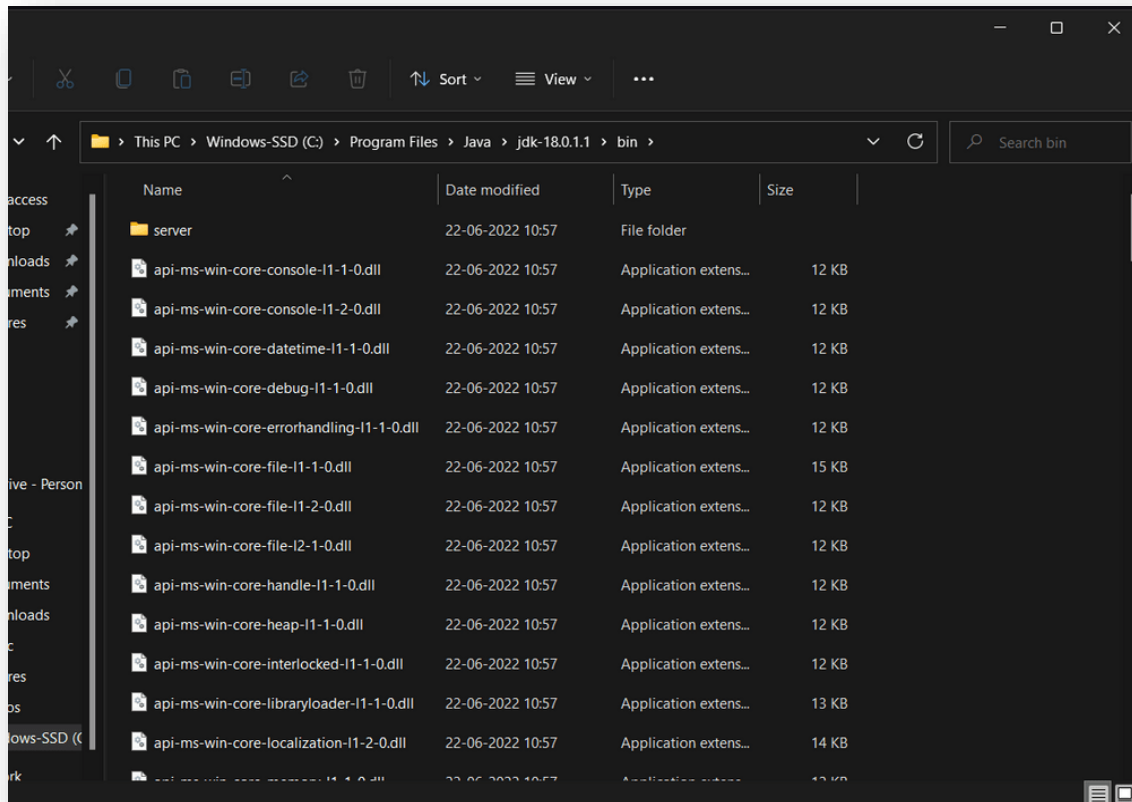


Step 2: Setup the Environment Variables

Once the installation gets completed, you need to configure environment variables to notify the system about the directory in which the **JDK files** are located.

Proceed to **C:\Program Files\Java\jdk- {YOUR_JDK_VERSION}\bin** (replace {-} with your JDK version)

Practical file of Object-oriented programming using JAVA

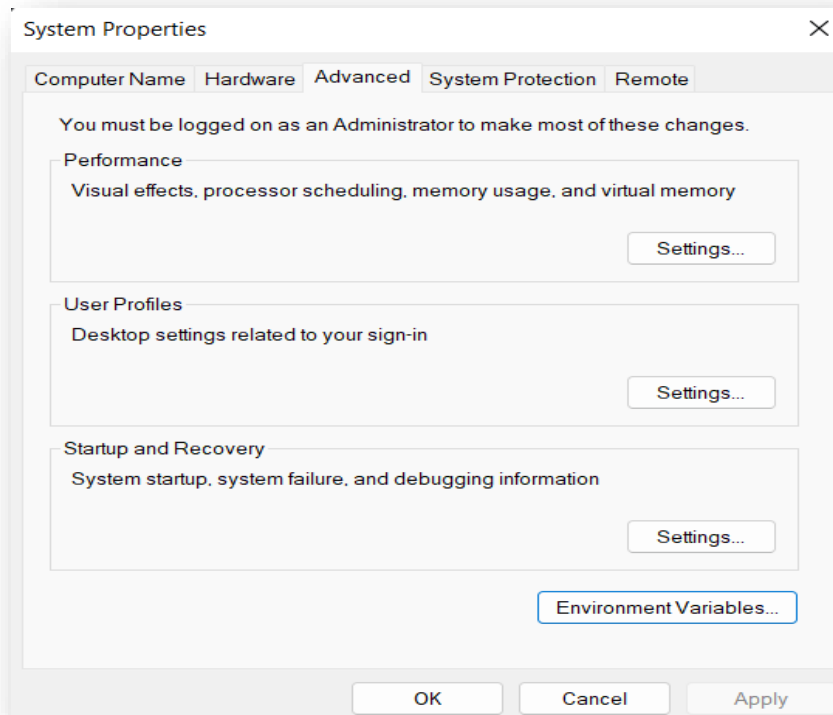


Step 2.1: To set the Environment Variables, you need to search Environment Variables in the [Task Bar](#) and click on “**Edit the system environment variables**”.

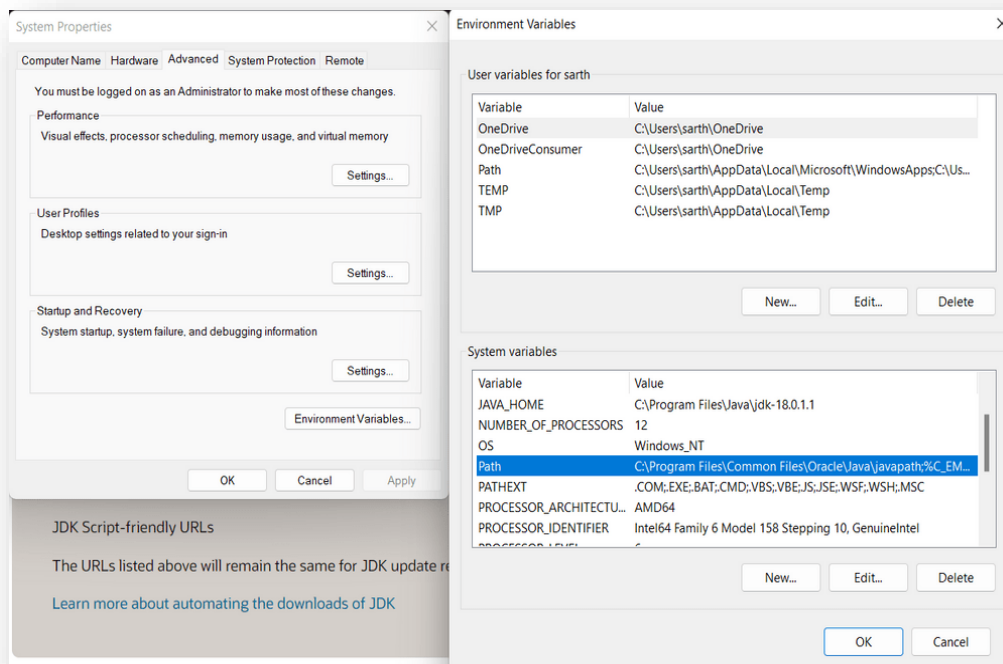
~ 5 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA



Step 2.2: Under the **Advanced** section, Click on “**Environment Variables**”.

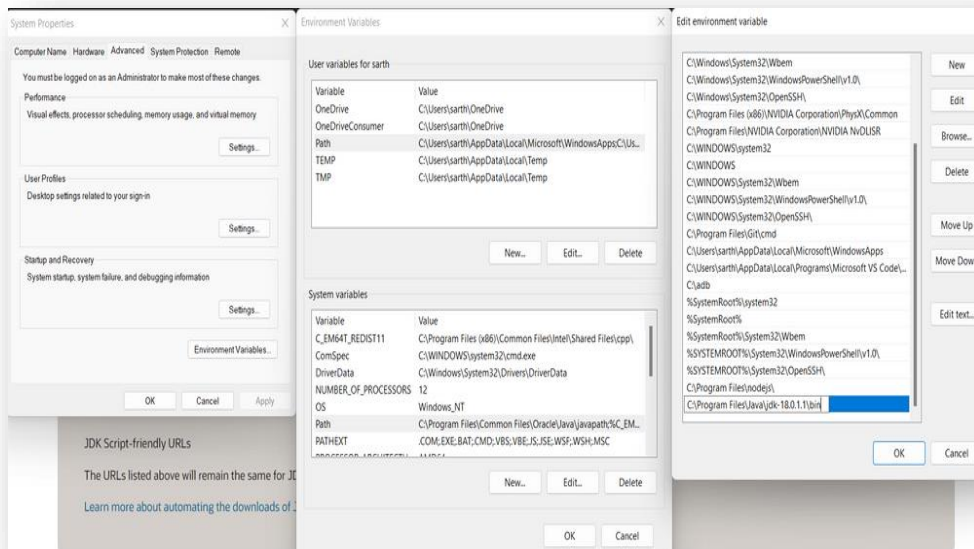


~ 6 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA

Step 2.3: Under **System variables**, select the “**Path**” variable and click on “**Edit**”. Click on “**New**” then paste the Path Address i.e. **C:\Program Files\Java\jdk- {YOUR_JDK_VERSION} \bin**. Click on “**OK**”.



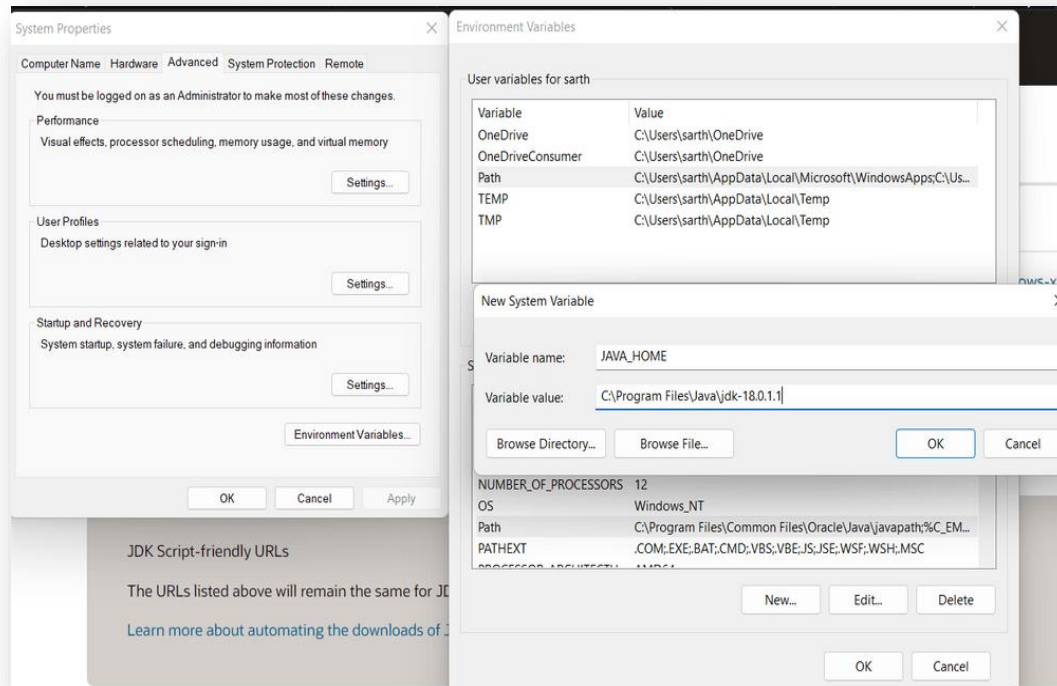
Step 2.4: Now, in the **Environment Variables** dialogue, under **System variables**, click on “**New**” and then under **Variable name:** **JAVA_HOME** and **Variable value:** paste address i.e.

C:\Program Files\Java\jdk-{YOUR_JDK_VERSION}. Click on **OK => OK => OK**.

~ 7 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA



Step 3: Check the Java Version

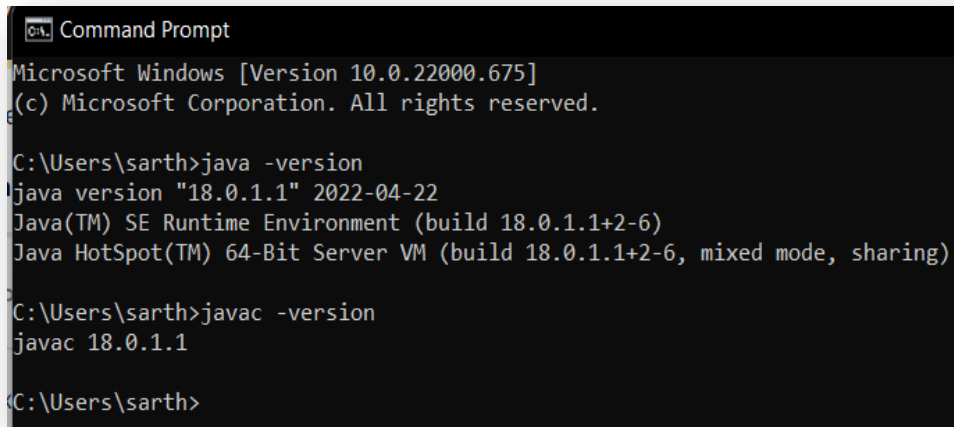
[Open Command Prompt](#) and enter the following commands:

```
java -version  
javac -version
```

~ 8 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA



```
Command Prompt
Microsoft Windows [Version 10.0.22000.675]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sarth>java -version
java version "18.0.1.1" 2022-04-22
Java(TM) SE Runtime Environment (build 18.0.1.1+2-6)
Java HotSpot(TM) 64-Bit Server VM (build 18.0.1.1+2-6, mixed mode, sharing)

C:\Users\sarth>javac -version
javac 18.0.1.1

C:\Users\sarth>
```

Install JDK on Linux

Follow the steps below to install the Java software Development Kit on a [Linux environment](#). These instructions are applicable to various Linux distributions such as Ubuntu, Fedora, and CentOS.

Step 1: Install Oracle Java Development Kit (JDK)

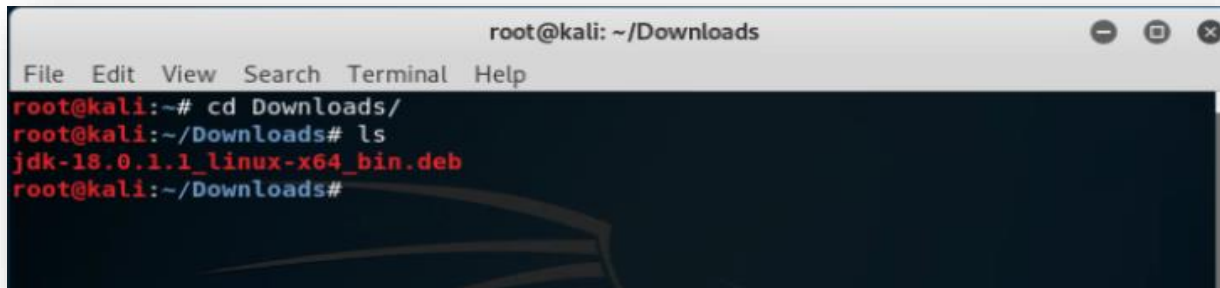
To install the downloaded JDK File using terminal, Open terminal and change your directory to downloads by using the command:

```
$ cd downloads
```

To list files and packages present in the directory, Type

```
$ ls
```

Practical file of Object-oriented programming using JAVA



```
root@kali: ~/Downloads
File Edit View Search Terminal Help
root@kali:~# cd Downloads/
root@kali:~/Downloads# ls
jdk-18.0.1.1_linux-x64_bin.deb
root@kali:~/Downloads#
```

Step 1.1: Now we use **Debian Package Manager** to configure our downloaded file for installation by typing

```
$ sudo dpkg -i jdk-{YOUR_JDK_VERSION} (replace {-} with your version)
```

Enter your password as we have run an elevated prompt, i.e. the **sudo** or **superuser do** command, which is a quick way to grant specific users permission to perform specific system commands at the system's root (most powerful) level.



```
ritikshrivas@kali-linux: ~/Downloads
(ritikshrivas@kali-linux)-[~]
$ cd Downloads
(ritikshrivas@kali-linux)-[~/Downloads]
$ sudo dpkg -i jdk-18 linux-x64 bin.deb
[sudo] password for ritikshrivas: 
```

Step 1.2: Now, Type the following commands to proceed with the installation

Practical file of Object-oriented programming using JAVA

```
$ sudo update-alternatives --install /usr/bin/java java /usr/lib/jvm/jdk-  
{YOUR_JDK_VERSION}/bin/java 1  
$ sudo update-alternatives --install /usr/bin/javac javac /usr/lib/jvm/jdk-  
{YOUR_JDK_VERSION}/bin/javac 1
```

Step 3: Configure JAVA_HOME Environment Variable

After the installation is complete, we have to configure environment variables to notify the system about the directory in which jdk files are located. To find the Location of the JDK Files, run this command and copy the File Location.

```
$ sudo update-alternatives --config java
```

Step 3.1: Proceed to add **JAVA_HOME=" /usr/lib/jvm/jdk-{YOUR_JDK_VERSION}"**

Proceed to save and close the file.



Step 3.2: Now we have to refresh the environment file by using this command:

Practical file of Object-oriented programming using JAVA

```
$ SOURCE /etc/environment
```

And echo the JAVA_HOME Path

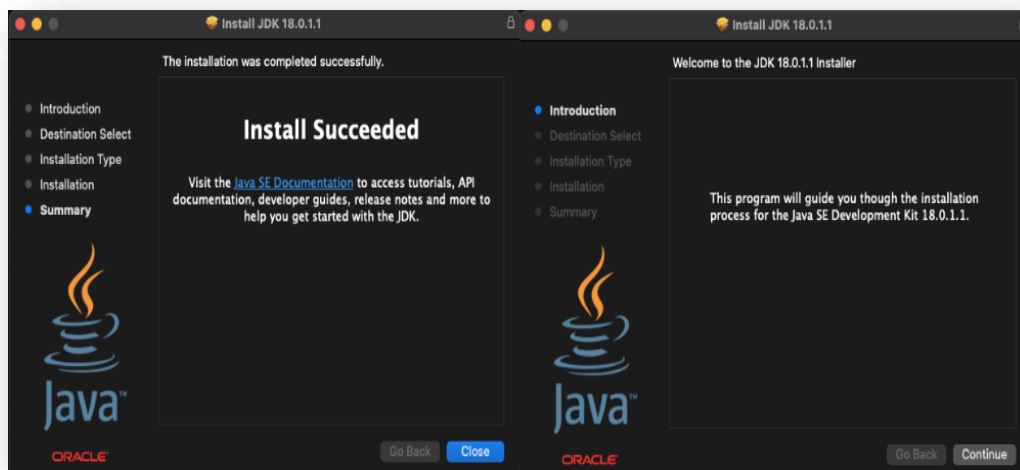
```
$ echo $JAVA_HOME
```

Install JDK on macOS

Follow the steps below to install the [Java Development Kit](#) (JDK) on macOS. These instructions are compatible with various versions of macOS, including **Mojave**, **Catalina**, **Big Sur**, **Monterey**, **Ventura**, **Sonoma** the latest macOS **Sequoia**.

Step 1: Install Oracle Java Development Kit (JDK)

Proceed to Install the JDK kit by following the bootstrapped steps:



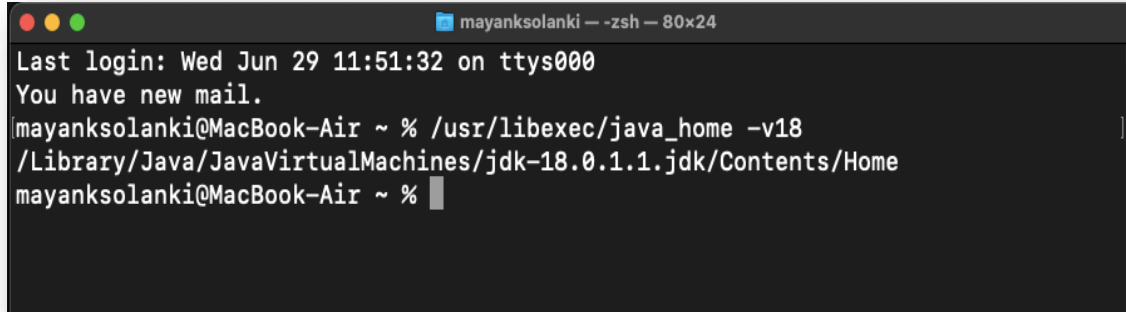
Step 2: Configure environment variables

~ 12 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA

Open the terminal and pass the following commands. To find the Location of the JAVA_HOME, run this command:

A terminal window titled 'mayanksolanki — zsh — 80x24'. It shows the output of a command to find the location of the Java Virtual Machine (JDK) installation. The text in the terminal is: 'Last login: Wed Jun 29 11:51:32 on ttys000', 'You have new mail.', 'mayanksolanki@MacBook-Air ~ % /usr/libexec/java_home -v18', '/Library/Java/JavaVirtualMachines/jdk-18.0.1.1.jdk/Contents/Home', and 'mayanksolanki@MacBook-Air ~ %' with a cursor.

```
Last login: Wed Jun 29 11:51:32 on ttys000
You have new mail.
mayanksolanki@MacBook-Air ~ % /usr/libexec/java_home -v18
/Library/Java/JavaVirtualMachines/jdk-18.0.1.1.jdk/Contents/Home
mayanksolanki@MacBook-Air ~ %
```

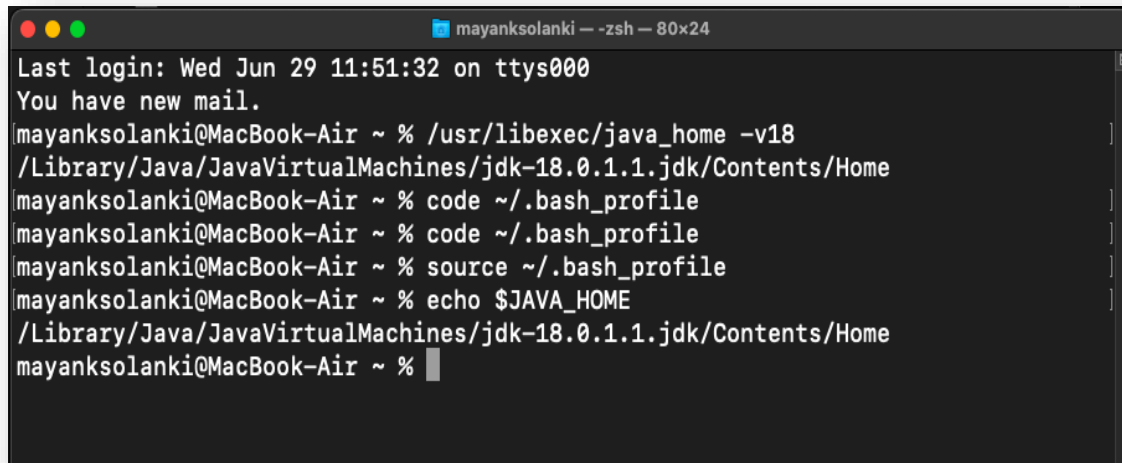
Step 2.2: At the very bottom, we have to export the path we obtained earlier i.e.

A screenshot of a text editor window titled '.bash_profile'. The window shows a single line of code: '1 export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-18.0.1.1.jdk/Contents/Home'. The text is highlighted in a light blue color. The window has a dark background and a light-colored border.

```
.bash_profile
1 export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-18.0.1.1.jdk/Contents/Home
```

Step 2.3: Now we have to refresh the environment file by using this command

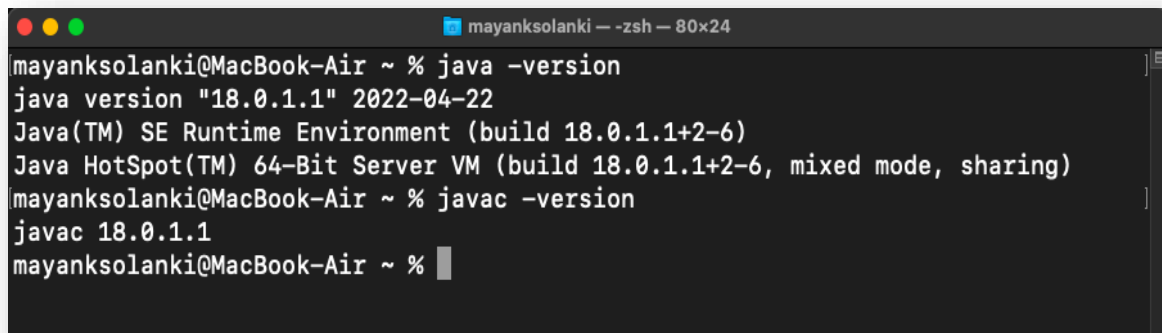
Practical file of Object-oriented programming using JAVA



```
mayanksolanki — zsh — 80x24
Last login: Wed Jun 29 11:51:32 on ttys000
You have new mail.
mayanksolanki@MacBook-Air ~ % /usr/libexec/java_home -v18
/Library/Java/JavaVirtualMachines/jdk-18.0.1.1.jdk/Contents/Home
mayanksolanki@MacBook-Air ~ % code ~/.bash_profile
mayanksolanki@MacBook-Air ~ % code ~/.bash_profile
mayanksolanki@MacBook-Air ~ % source ~/.bash_profile
mayanksolanki@MacBook-Air ~ % echo $JAVA_HOME
/Library/Java/JavaVirtualMachines/jdk-18.0.1.1.jdk/Contents/Home
mayanksolanki@MacBook-Air ~ %
```

Step 3: Verify the Java Version

In the **terminal**, enter the following commands



```
mayanksolanki — zsh — 80x24
mayanksolanki@MacBook-Air ~ % java -version
java version "18.0.1.1" 2022-04-22
Java(TM) SE Runtime Environment (build 18.0.1.1+2-6)
Java HotSpot(TM) 64-Bit Server VM (build 18.0.1.1+2-6, mixed mode, sharing)
mayanksolanki@MacBook-Air ~ % javac -version
javac 18.0.1.1
mayanksolanki@MacBook-Air ~ %
```

Compiling simple Java Program: -

Practical file of Object-oriented programming using JAVA

```
// This is a simple Java program.  
// FileName : "HelloWorld.java".  
  
class HelloWorld {  
    // Your program begins with a call to main().  
    // Prints "Hello, World" to the terminal window.  
    public static void main(String[] args)  
    {  
        System.out.println("Hello, World");  
    }  
}
```

Output

Hello, World

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.2

Aim: Programming exercise on control flow statement, operators and looping statements in Java

Control flow statement:

Exercise 1: If – Else Statement

Write a Java program that checks if a number is even or odd

Input: -

```
import java. util. Scanner;

public class Main {
    public static void main (String [] args) {

        Scanner scanner = new Scanner (System.in);

        System.out.print("Enter a number: ");
        int number = scanner. nextInt ();

        if (number % 2 == 0) {
            System.out.println(number + " is even.");
        } else {
            System.out.println(number + " is odd.");
        }

        scanner. Close ();
    }
}
```

~ 16 ~

Practical file of Object-oriented programming using JAVA

Output:

```
Enter a number: 23
23 is odd.

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 2: Switch Statement

Write a Java program that takes a day number (1-7) and prints the corresponding day of the week.

Input:

```
import java. util. Scanner;

public class Main {
    public static void main (String [] args) {
        Scanner scanner = new Scanner (System.in);

        System.out.print("Enter a day number (1-7): ");
        int day Number = scanner. nextInt ();

        String day;

        switch (dayNumber) {
            case 1:
```

~ 17 ~

Practical file of Object-oriented programming using JAVA

```
        day = "Monday";
        break;
    case 2:
        day = "Tuesday";
        break;
    case 3:
        day = "Wednesday";
        break;
    case 4:
        day = "Thursday";
        break;
    case 5:
        day = "Friday";
        break;
    case 6:
        day = "Saturday";
        break;
    case 7:
        day = "Sunday";
        break;
    default:
        day = "Invalid day number! Please enter a number between 1 and 7.";
        break;
}

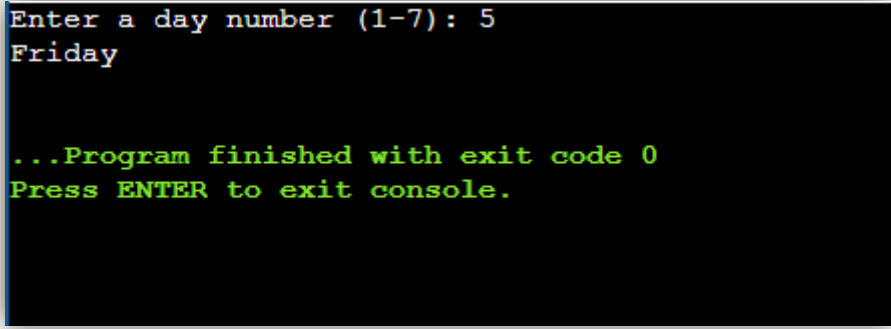
System.out.println(day);

scanner.close();
```

Practical file of Object-oriented programming using JAVA

```
}  
}
```

Output:



```
Enter a day number (1-7): 5  
Friday  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Exercise 3: Arithmetic and Relational operators

Write a Java Program that takes two numbers and performs addition, subtraction, multiplication, and division. Also, check if the first number is greater than the second

```
import java. util. Scanner;
```

```
public class Main {  
    public static void main (String [] args) {  
        Scanner scanner = new Scanner (System.in);  
  
        System.out.print("Enter first number: ");  
        int num1 = scanner. nextInt();  
  
        System.out.print("Enter second number: ");  
        int num2 = scanner.nextInt();  
  
        System.out.println("Addition: " + (num1 + num2));  
        System.out.println("Subtraction: " + (num1 - num2));
```

~ 19 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA

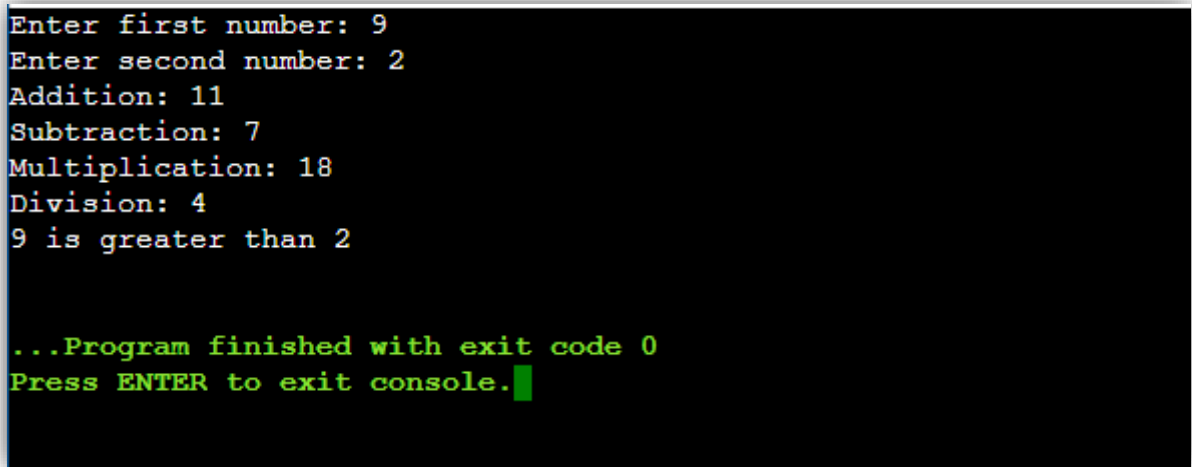
```
System.out.println("Multiplication: " + (num1 * num2));

if (num2 != 0) {
    System.out.println("Division: " + (num1 / num2));
} else {
    System.out.println("Division: Undefined (cannot divide by zero)");
}

if (num1 > num2) {
    System.out.println(num1 + " is greater than " + num2);
} else if (num1 < num2) {
    System.out.println(num1 + " is less than " + num2);
} else {
    System.out.println(num1 + " is equal to " + num2);
}

scanner.close ();
}
}
```

Output: -



```
Enter first number: 9
Enter second number: 2
Addition: 11
Subtraction: 7
Multiplication: 18
Division: 4
9 is greater than 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Practical file of Object-oriented programming using JAVA

Looping Statements

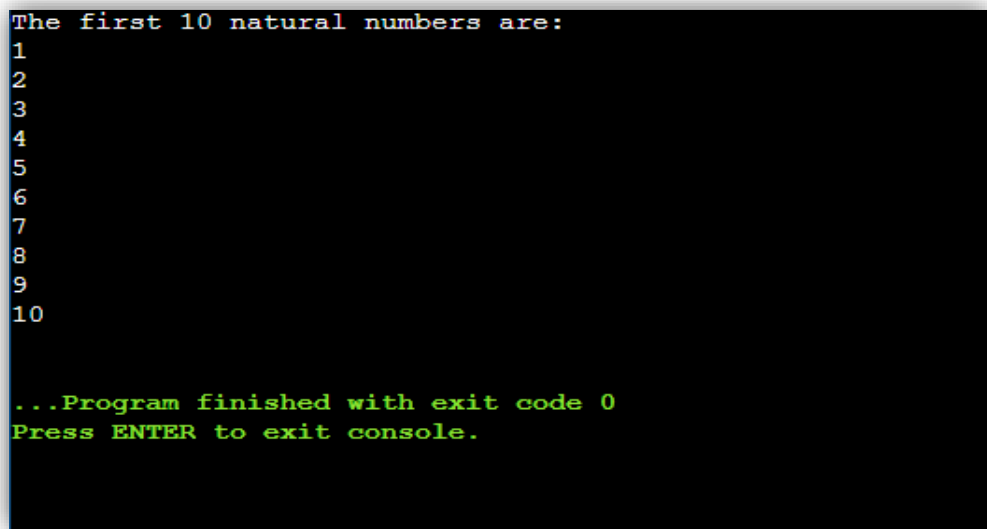
Exercise 4: For Loop

Write a Java Program that prints the first 10 natural numbers using for loop.

Input:

```
public class Main {  
    public static void main (String [] args) {  
  
        System.out.println("The first 10 natural numbers are:");  
  
        for (int i = 1; i <= 10; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

Output:



```
The first 10 natural numbers are:  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Practical file of Object-oriented programming using JAVA

Exercise 5: While Loop:

Write a Java Program that takes a number from the user and prints the multiplication table of that number using a While loop.

Input:

```
import java. util. Scanner;

public class Main {
    public static void main (String [] args) {

        Scanner scanner = new Scanner (System.in);

        System.out.print("Enter a number to print its multiplication table: ");
        int number = scanner. nextInt ();

        int i = 1;
        while (i <= 10) {
            System.out.println(number + " x " + i + " = " + (number * i));
            i++;
        }

        scanner. close ();
    }
}
```

Practical file of Object-oriented programming using JAVA

Output:

```
Enter a number to print its multiplication table: 23
23 x 1 = 23
23 x 2 = 46
23 x 3 = 69
23 x 4 = 92
23 x 5 = 115
23 x 6 = 138
23 x 7 = 161
23 x 8 = 184
23 x 9 = 207
23 x 10 = 230

...Program finished with exit code 0
Press ENTER to exit console.□
```

~ 23 ~

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.3

Aim: Program to scan the input using scanner class

Theory:

Methods commonly used in the scanner class:

1. nextln ()- Read an integer.
2. nextDouble ()- Read a double.
3. nextLine ()- Reads a string (including spaces).
4. next ()- Read a Single word (upto a space).
5. nextBoolean ()- Reads a Boolean (true or flase).

Program:

Input:

```
import java. util. Scanner;

public class Main {

    public static void main (String [] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter an integer\n ");
        int intValue = scanner.nextInt();

        System.out.print("Enter a float value\n ");
        float floatValue = scanner.nextFloat();

        System.out.print("Enter a double value\n ");
        double doubleValue = scanner.nextDouble();

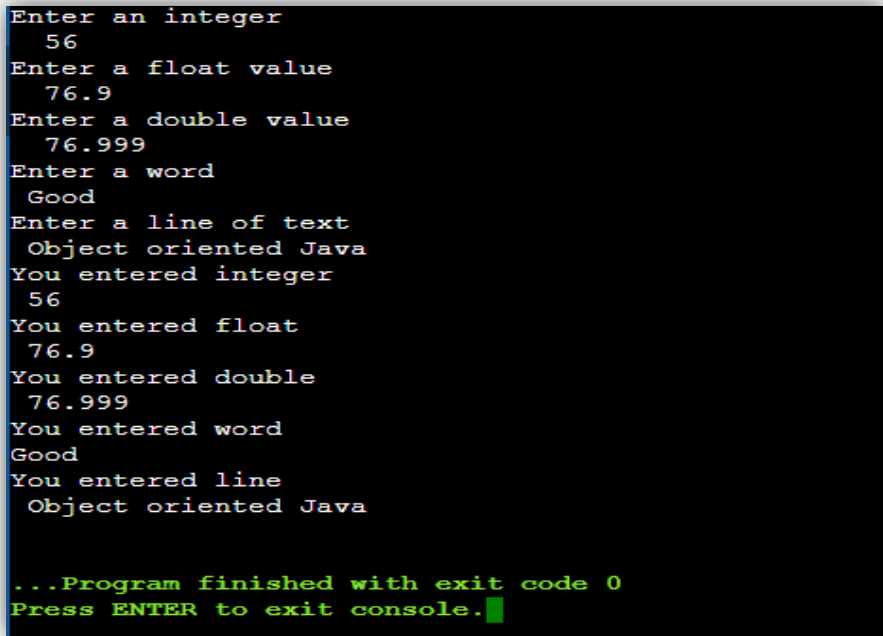
        System.out.print("Enter a word\n ");
        String word = scanner.next();
```

~ 24 ~

Practical file of Object-oriented programming using JAVA

```
scanner.nextLine();  
System.out.print("Enter a line of text\n ");  
String line = scanner. next Line ();  
  
System.out.println("You entered integer\n " + intValue);  
System.out.println("You entered float\n" + floatValue);  
System.out.println("You entered double\n" + doubleValue);  
System.out.println("You entered word\n " + word);  
System.out.println("You entered line: " + line);  
scanner. close ();  
}  
}
```

Output:



```
Enter an integer  
56  
Enter a float value  
76.9  
Enter a double value  
76.999  
Enter a word  
Good  
Enter a line of text  
Object oriented Java  
You entered integer  
56  
You entered float  
76.9  
You entered double  
76.999  
You entered word  
Good  
You entered line  
Object oriented Java  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.4

Aim: Programming exercise on arrays and functions in Java

Theory:

- 1. Arrays:** An array is a collection of elements of the same type, stored in contiguous memory locations. It allows you to store multiple values in a single variable.
- 2. Functions:** Functions are blocks of code that perform a specific task and can be reused throughout the program.

Program:

Input:

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void displayArray(int[] arr) {  
        System.out.println("Array elements are:");  
        for (int num : arr) {  
            System.out.print(num + " ");  
        }  
        System.out.println();  
    }
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        System.out.print("Enter the size of the array: ");
```

```
        int size = scanner.nextInt();
```

~ 26 ~

Practical file of Object-oriented programming using JAVA

```
int[] numbers = new int[size];
```

```
System.out.println("Enter " + size + " elements:");
```

```
for (int i = 0; i < size; i++) {
```

```
    numbers[i] = scanner.nextInt();
```

```
}
```

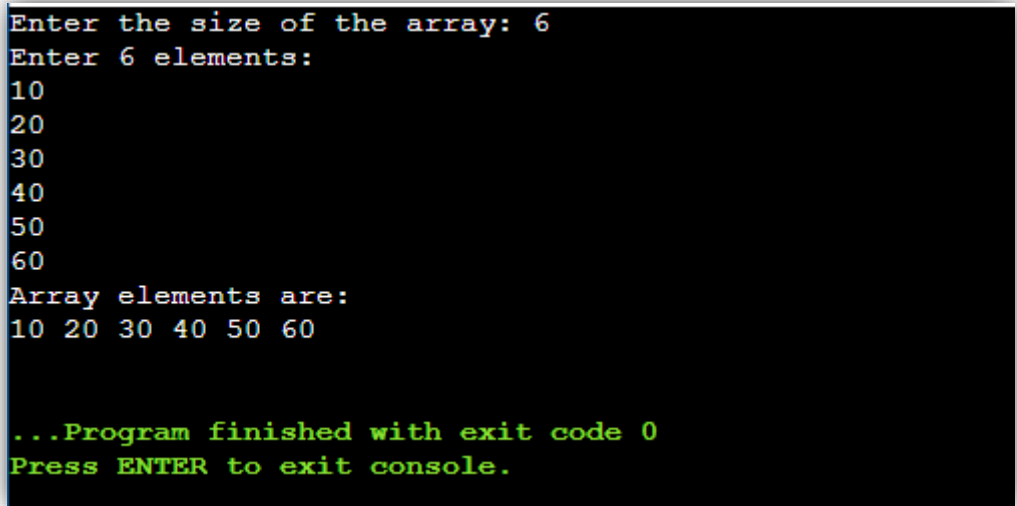
```
displayArray(numbers);
```

```
scanner.close();
```

```
}
```

```
}
```

Output:



```
Enter the size of the array: 6
Enter 6 elements:
10
20
30
40
50
60
Array elements are:
10 20 30 40 50 60

...Program finished with exit code 0
Press ENTER to exit console.
```

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.5

Aim: Program to demonstrate the concept of classes and objects using access specifiers.

Theory:

1. **Class:** A blueprint for creating objects. It defines properties and methods that the objects created from the class can have.
2. **Object:** An instance of a class. It represents a specific entity with the properties and behaviors defined by the class.
3. **Access Specifiers:**
 - **Public:** Members declared as public can be accessed from any other class.
 - **Private:** Members declared as private can only be accessed within the same class.
 - **Protected:** Members declared as protected can be accessed within the same package and by subclasses.

Program:

Input:

```
import java.util.Scanner;

class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void displayInfo () {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

~ 28 ~

Practical file of Object-oriented programming using JAVA

```
}  
protected void greet() {  
    System.out.println("Hello, " + name + "!");  
}  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Enter your name: ");  
        String name = scanner.nextLine();  
  
        System.out.print("Enter your age: ");  
        int age = scanner.nextInt();  
  
        Person person = new Person(name, age);  
  
        person.displayInfo();  
        person.greet();  
  
        scanner.close();  
    }  
}
```

Practical file of Object-oriented programming using JAVA

Output:

```
Enter your name: Rohit
Enter your age: 19
Name: Rohit
Age: 19
Hello, Rohit!

...Program finished with exit code 0
Press ENTER to exit console.
```

~ 30 ~

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.6

Aim: Program to demonstrate the use of construction.

Theory:

A constructor is a special method that is called when an object is instantiated. It has the same name as the class and does not have a return type. Constructors are used to initialize the object's attributes.

Features of constructors:

1. Same name as the class.
2. No return type, not even void.

Types of constructors in Java:

1. **Default Constructor:** Provided by the compiler or defined explicitly without parameters.
2. **Parameterized Constructor:** Accepts arguments to initialize the object with specific values.
3. **Copy constructor:** Creates a new object by copying values from another object.

Program:

Input:

```
class Car {  
    String model;  
    int year;  
    public Car() {  
        model = "unknown";  
        year = 0;  
        System.out.println("Default Constructor Called");  
    }  
    public Car(String model, int year) {  
        this.model = model;
```


Practical file of Object-oriented programming using JAVA

```
this.year = year;

System.out.println("Parameterized Constructor Called");

}

public void displayDetails() {

    System.out.println("Model: " + model);

    System.out.println("Year: " + year);

}

}

public class Main {

    public static void main (String [] args) {

        Car car1 = new Car();

        car1.displayDetails();

        System.out.println();

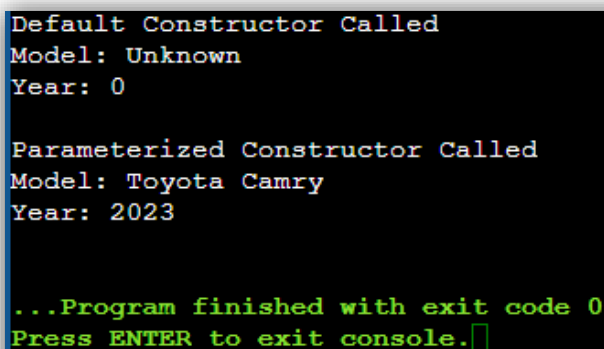
        Car car2 = new Car ("Toyota Camry", 2023);

        car2.displayDetails();

    }

}
```

Output:

A screenshot of a Java program's output in a console window. The text is displayed in a monospaced font with syntax highlighting: keywords like 'Default Constructor Called', 'Parameterized Constructor Called', and 'Program finished' are in green, while other text is in white. The output shows the execution of two Car objects: one with default values (Unknown model, Year 0) and one with specific values (Toyota Camry, Year 2023). The console ends with a message to press ENTER to exit.

```
Default Constructor Called
Model: Unknown
Year: 0

Parameterized Constructor Called
Model: Toyota Camry
Year: 2023

...Program finished with exit code 0
Press ENTER to exit console.
```

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.7

Aim: Programming exercise on difference types of inheritance in Java.

Theory: Inheritance is mechanism in Java where one class (subclass or child class) Inherits the properties and behaviors(methods) of another class (superclass or parent class). This promotes code reusability and establishes a relationship between classes.

Types of Inheritance:

1. **Single Inheritance:** A class inherits from one superclass.
2. **Multi-level Inheritance:** A class inherit from a single superclass.
3. **Hierarchical Inheritance:** Multiple classes inherit from a single superclass.

Program:

Input:

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}  
  
class Puppy extends Dog {  
    void weep() {  
        System.out.println("Puppy is weeping");  
    }  
}
```

~ 33 ~

Practical file of Object-oriented programming using JAVA

```
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat is meowing");  
    }  
}
```

```
interface Bird {  
    void fly ();  
}
```

```
interface Fish {  
    void swim ();  
}
```

```
class Duck implements Bird, Fish {  
    public void fly() {  
        System.out.println("Duck is flying");  
    }  
  
    public void swim() {  
        System.out.println("Duck is swimming");  
    }  
}
```

```
public class Main {  
    public static void main (String [] args) {  
  
        Dog dog = new Dog ();  
        dog.eat ();  
    }  
}
```

Practical file of Object-oriented programming using JAVA

```
dog. bark ();
```

```
Puppy puppy = new Puppy();
```

```
puppy.eat();
```

```
puppy.bark();
```

```
puppy.weep();
```

```
Cat cat = new Cat();
```

```
cat.eat();
```

```
cat.meow();
```

```
Duck duck = new Duck ();
```

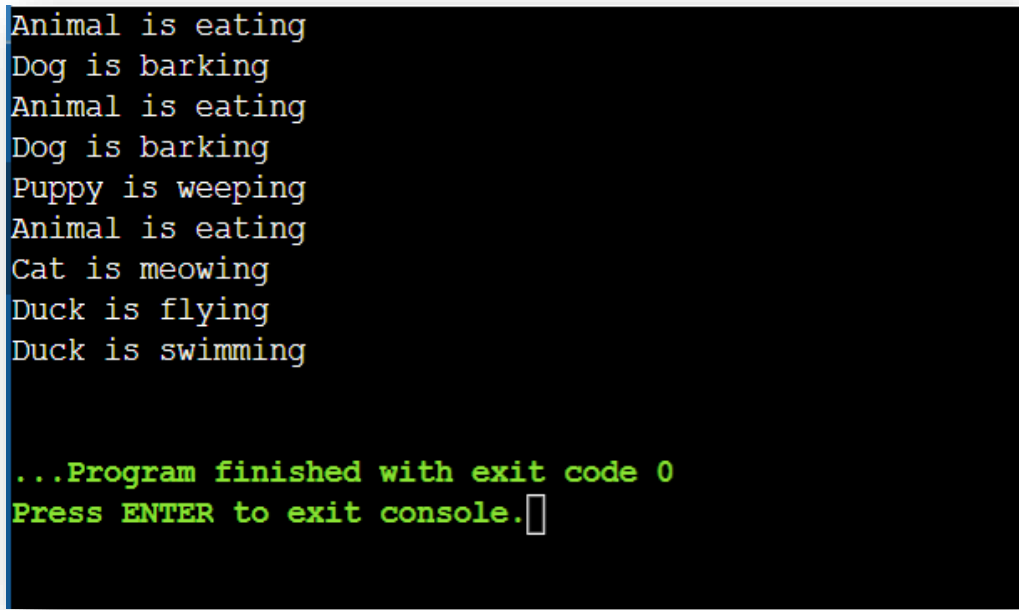
```
duck.fly();
```

```
duck.swim();
```

```
}
```

```
}
```

Output:

A screenshot of a Java program's output in a console window. The text is displayed on a black background with a blue border on the left. The output shows the execution of various methods: 'Animal is eating', 'Dog is barking', 'Puppy is weeping', 'Cat is meowing', 'Duck is flying', and 'Duck is swimming'. At the bottom, it states '...Program finished with exit code 0' and 'Press ENTER to exit console.' with a cursor icon.

```
Animal is eating  
Dog is barking  
Animal is eating  
Dog is barking  
Puppy is weeping  
Animal is eating  
Cat is meowing  
Duck is flying  
Duck is swimming  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.8

Aim: Program to demonstrate the concept of overloading and overriding

Theory:

- 1. Method Overloading:** This occurs when multiple methods in the same class have the same name but different parameters (difference type, number, or both). It allows methods to perform similar functions with different types of input.
- 2. Method Overriding:** This occurs When a subclass provides a specific implementation of a method that is already defined in its superclass. It allows a subclass to provide a specific behavior for a method that is already defined in its parent class.

Program:

Input:

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    void display() {  
        System.out.println("This is the Calculator class");  
    }  
}  
  
class AdvancedCalculator extends Calculator {
```

~ 36 ~

Practical file of Object-oriented programming using JAVA

```
@Override
void display() {
    System.out.println("This is the Advanced Calculator class");
}

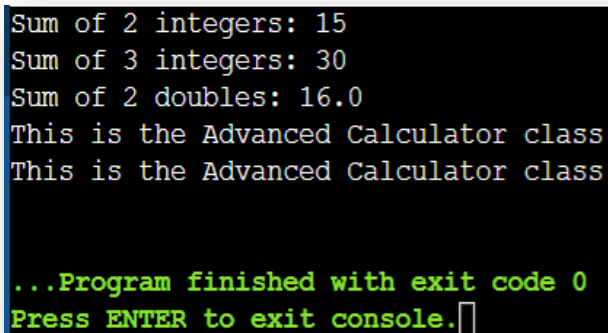
public class Main {
    public static void main(String[] args) {

        Calculator calc = new Calculator();
        System.out.println("Sum of 2 integers: " + calc.add(5, 10));
        System.out.println("Sum of 3 integers: " + calc.add(5, 10, 15));
        System.out.println("Sum of 2 doubles: " + calc.add(5.5, 10.5));

        AdvancedCalculator advCalc = new AdvancedCalculator();
        advCalc.display();

        Calculator parentRef = new AdvancedCalculator();
        parentRef.display();
    }
}
```

Output:

A screenshot of a Java program's output in a console window. The output shows the results of arithmetic operations and class instantiation. The text is as follows:

```
Sum of 2 integers: 15
Sum of 3 integers: 30
Sum of 2 doubles: 16.0
This is the Advanced Calculator class
This is the Advanced Calculator class

...Program finished with exit code 0
Press ENTER to exit console.
```

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.9

Aim: Program to demonstrate the concept of packages, abstract classes and interface.

Theory:

- 1. Packages:** A package is a namespace that organizes a set of related classes and interfaces. Conceptually similar to directories on your computer, packages help avoid conflicts and can control access with protected and default access levels.
- 2. Abstract Classes:** An abstract class is a class that cannot be instantiated on its own and may contain abstract methods (method without a body). Subclasses must provide implementations for these abstract methods.
- 3. Interfaces:** An interface is a reference type in Java that is similar to a class but can only contains constants, method signatures, default methods, static methods, and nested types. Interfaces cannot contain instance fields. A class implements an interface, providing the body for all the methods defined in the interface.

Program:

Input:

```
interface Shape {  
    double area ();  
}  
  
abstract class AbstractShape implements Shape {  
    double dimension1;  
    double dimension2;  
  
    AbstractShape(double d1, double d2) {  
        this.dimension1 = d1;  
        this.dimension2 = d2;  
    }  
}
```

~ 38 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA

```
class Rectangle extends AbstractShape {
    Rectangle(double length, double width) {
        super(length, width);
    }

    public double area() {
        return dimension1 * dimension2;
    }
}

class Circle extends AbstractShape {
    Circle(double radius) {
        super(radius, radius);
    }

    public double area() {
        return Math.PI * dimension1 * dimension1;
    }
}

public class main {
    public static void main(String[] args) {

        Shape rectangle = new Rectangle(5, 10);
        System.out.println("Rectangle Area: " + rectangle.area());

        Shape circle = new Circle(7);

        System.out.println("Circle Area: " + circle.area());
    }
}
```

~ 39 ~

Name: Jasmine
CSE (4th semester)
Registration no. 231949508520

Practical file of Object-oriented programming using JAVA

Output:

```
Rectangle Area: 50.0  
Circle Area: 153.93804002589985  
  
...Program finished with exit code 0  
Press ENTER to exit console.█
```

~ 40 ~

Practical file of Object-oriented programming using JAVA

PRACTICAL NO.10

Aim: Programming exercise on exception handling.

Theory:

- 1. Exception:** An exception is an event that disrupts the normal flow of a program's execution. It can occur due to various reasons, such as invalid input, file not found, or network issues.
- 2. Try – Catch Block:** A try – catch block is used to handle exceptions. The code that might throw an exception is placed inside the try block, and the handling code is placed inside the catch block.
- 3. Finally Block:** The finally block is used to execute code after the try-catch block, regardless of whether an exception was thrown or not. It is typically used for cleanup activities.
- 4. Throwing Exceptions:** You can throw exceptions manually using the throw keyword.

Program:

Input:

```
import java.util.Scanner;

public class Main{

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        try {

            System.out.print("Enter the first number: ");

            int num1 = scanner.nextInt();
```

~ 41 ~

Practical file of Object-oriented programming using JAVA

```
System.out.print("Enter the second number: ");

int num2 = scanner.nextInt();

int result = num1 / num2;

System.out.println("Result: " + result);

} catch (ArithmeticException e) {

    System.out.println("Error: Cannot divide by zero!");

} catch (java.util.InputMismatchException e) {

    System.out.println("Error: Invalid input. Please enter integers only.");

} finally {

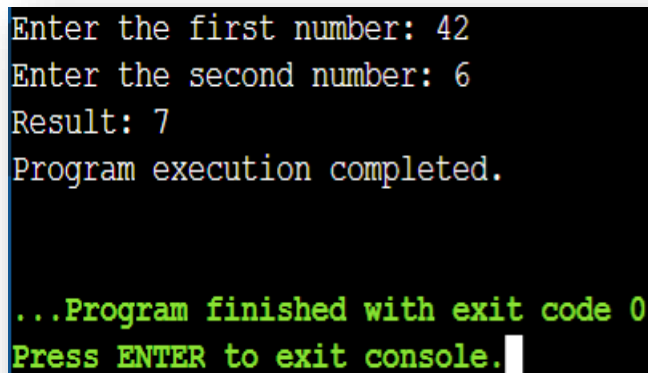
    System.out.println("Program execution completed.");

    scanner.close();

}

}
```

Output:

A screenshot of a console window with a black background and white text. The text shows the program's execution: it prompts for the first number (42) and the second number (6), calculates the result (7), and prints "Program execution completed." Below this, in green text, it says "...Program finished with exit code 0" and "Press ENTER to exit console." with a white cursor at the end.

```
Enter the first number: 42
Enter the second number: 6
Result: 7
Program execution completed.

...Program finished with exit code 0
Press ENTER to exit console.
```